

- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapters 2, 7, and 14.
- Deuffhard, P. 2004, *Newton Methods for Nonlinear Problems* (Berlin: Springer).
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), Chapter 8.
- Householder, A.S. 1970, *The Numerical Treatment of a Single Nonlinear Equation* (New York: McGraw-Hill).

9.1 Bracketing and Bisection

We will say that a root is *bracketed* in the interval (a, b) if $f(a)$ and $f(b)$ have opposite signs. If the function is continuous, then at least one root must lie in that interval (the *intermediate value theorem*). If the function is discontinuous, but bounded, then instead of a root there might be a step discontinuity that crosses zero (see Figure 9.1.1). For numerical purposes, that might as well be a root, since the behavior is indistinguishable from the case of a continuous function whose zero crossing occurs in between two “adjacent” floating-point numbers in a machine’s finite-precision representation. Only for functions with singularities is there the possibility that a bracketed root is not really there, as for example

$$f(x) = \frac{1}{x - c} \quad (9.1.1)$$

Some root-finding algorithms (e.g., bisection in this section) will readily converge to c in (9.1.1). Luckily there is not much possibility of your mistaking c , or any number x close to it, for a root, since mere evaluation of $|f(x)|$ will give a very large, rather than a very small, result.

If you are given a function in a black box, there is no sure way of bracketing its roots, or even of determining that it has roots. If you like pathological examples, think about the problem of locating the two real roots of equation (3.0.1), which dips below zero only in the ridiculously small interval of about $x = \pi \pm 10^{-667}$.

In the next chapter we will deal with the related problem of bracketing a function’s minimum. There it is possible to give a procedure that always succeeds; in essence, “Go downhill, taking steps of increasing size, until your function starts back uphill.” There is no analogous procedure for roots. The procedure “go downhill until your function changes sign,” can be foiled by a function that has a simple extremum. Nevertheless, if you are prepared to deal with a “failure” outcome, this procedure is often a good first start; success is usual if your function has opposite signs in the limit $x \rightarrow \pm\infty$.

```
template <class T>
```

```
Bool zbrac(T &func, Doub &x1, Doub &x2)
```

Given a function or functor `func` and an initial guessed range `x1` to `x2`, the routine expands the range geometrically until a root is bracketed by the returned values `x1` and `x2` (in which case `zbrac` returns `true`) or until the range becomes unacceptably large (in which case `zbrac` returns `false`).

```
{
```

```
    const Int NTRY=50;
```

```
    const Doub FACTOR=1.6;
```

roots.h

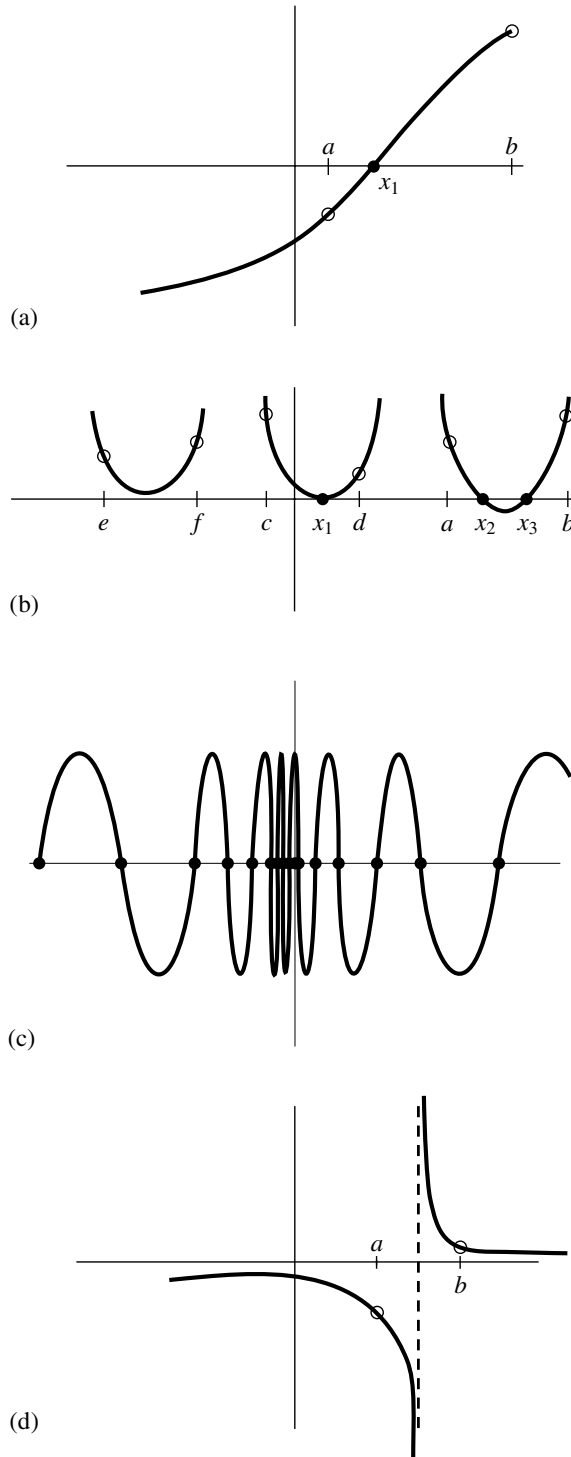


Figure 9.1.1. Some situations encountered while root finding: (a) an isolated root x_1 bracketed by two points a and b at which the function has opposite signs; (b) there is not necessarily a sign change in the function near a double root (in fact, there is not necessarily a root!); (c) a pathological function with many roots; in (d) the function has opposite signs at points a and b , but the points bracket a singularity, not a root.

```

    if (x1 == x2) throw("Bad initial range in zbrac");
    Doub f1=func(x1);
    Doub f2=func(x2);
    for (Int j=0;j<NTRY;j++) {
        if (f1*f2 < 0.0) return true;
        if (abs(f1) < abs(f2))
            f1=func(x1 += FACTOR*(x1-x2));
        else
            f2=func(x2 += FACTOR*(x2-x1));
    }
    return false;
}

```

Alternatively, you might want to “look inward” on an initial interval, rather than “look outward” from it, asking if there are any roots of the function $f(x)$ in the interval from x_1 to x_2 when a search is carried out by subdivision into n equal intervals. The following function calculates brackets for distinct intervals that each contain one or more roots.

```

template <class T>
void zbrak(T &fx, const Doub x1, const Doub x2, const Int n, VecDoub_0 &xb1,
          VecDoub_0 &xb2, Int &nroot)

```

roots.h

Given a function or functor fx defined on the interval $[x_1, x_2]$, subdivide the interval into n equally spaced segments, and search for zero crossings of the function. $nroot$ will be set to the number of bracketing pairs found. If it is positive, the arrays $xb1[0..nroot-1]$ and $xb2[0..nroot-1]$ will be filled sequentially with any bracketing pairs that are found. On input, these vectors may have any size, including zero; they will be resized to $\geq nroot$.

```

{
    Int nb=20;
    xb1.resize(nb);
    xb2.resize(nb);
    nroot=0;
    Doub dx=(x2-x1)/n;
    Doub x=x1;
    Doub fp=fx(x1);
    for (Int i=0;i<n;i++) {
        Doub fc=fx(x += dx);
        if (fc*fp <= 0.0) {
            xb1[nroot]=x-dx;
            xb2[nroot++]=x;
            if(nroot == nb) {
                VecDoub tempvec1(xb1),tempvec2(xb2);
                xb1.resize(2*nb);
                xb2.resize(2*nb);
                for (Int j=0; j<nb; j++) {
                    xb1[j]=tempvec1[j];
                    xb2[j]=tempvec2[j];
                }
                nb *= 2;
            }
        }
        fp=fc;
    }
}

```

Determine the spacing appropriate to the mesh.

Loop over all intervals

If a sign change occurs, then record values for the bounds.

9.1.1 Bisection Method

Once we know that an interval contains a root, several classical procedures are available to refine it. These proceed with varying degrees of speed and sure-

ness toward the answer. Unfortunately, the methods that are guaranteed to converge plod along most slowly, while those that rush to the solution in the best cases can also dash rapidly to infinity without warning if measures are not taken to avoid such behavior.

The *bisection method* is one that cannot fail. It is thus not to be sneered at as a method for otherwise badly behaved problems. The idea is simple. Over some interval the function is known to pass through zero because it changes sign. Evaluate the function at the interval's midpoint and examine its sign. Use the midpoint to replace whichever limit has the same sign. After each iteration the bounds containing the root decrease by a factor of two. If after n iterations the root is known to be within an interval of size ϵ_n , then after the next iteration it will be bracketed within an interval of size

$$\epsilon_{n+1} = \epsilon_n/2 \quad (9.1.2)$$

neither more nor less. Thus, we know in advance the number of iterations required to achieve a given tolerance in the solution,

$$n = \log_2 \frac{\epsilon_0}{\epsilon} \quad (9.1.3)$$

where ϵ_0 is the size of the initially bracketing interval and ϵ is the desired ending tolerance.

Bisection *must* succeed. If the interval happens to contain more than one root, bisection will find one of them. If the interval contains no roots and merely straddles a singularity, it will converge on the singularity.

When a method converges as a factor (less than 1) times the previous uncertainty to the first power (as is the case for bisection), it is said to converge *linearly*. Methods that converge as a higher power,

$$\epsilon_{n+1} = \text{constant} \times (\epsilon_n)^m \quad m > 1 \quad (9.1.4)$$

are said to converge *superlinearly*. In other contexts, “linear” convergence would be termed “exponential” or “geometrical.” That is not too bad at all: Linear convergence means that successive significant figures are won linearly with computational effort.

It remains to discuss practical criteria for convergence. It is crucial to keep in mind that only a finite set of floating point values have exact computer representations. While your function might analytically pass through zero, it is probable that its computed value is never zero, for any floating-point argument. One must decide what accuracy on the root is attainable: Convergence to within 10^{-10} in absolute value is reasonable when the root lies near 1 but certainly unachievable if the root lies near 10^{26} . One might thus think to specify convergence by a relative (fractional) criterion, but this becomes unworkable for roots near zero. To be most general, the routines below will require you to specify an absolute tolerance, such that iterations continue until the interval becomes smaller than this tolerance in absolute units. Often you may wish to take the tolerance to be $\epsilon(|x_1| + |x_2|)/2$, where ϵ is the machine precision and x_1 and x_2 are the initial brackets. When the root lies near zero you ought to consider carefully what reasonable tolerance means for your function. The following routine quits after 50 bisections in any event, with $2^{-50} \approx 10^{-15}$.